Backtracking, Branch & Bound, and Randomized Algorithms

Assignment

Submitted By:

Rohit Sharma Pandey

M.E Computer 1st Semester

Submitted in Partial Fulfillment of the Requirement of Master's Degree in ME Computer

Submitted to:

Everest Engineering College

Sanepa, Lalitpur-2



September 2026

Acknowledgement

I would like to express my sincere gratitude to **Everest Engineering College**, affiliated to **Pokhara University** for providing me with an opportunity to pursue my Master's degree in **ME COMPUTER**. I am grateful to the faculty members who have imparted their knowledge and skills throughout my academic journey.

I extend my heartfelt thanks to **Er. Narayan Sapkota**, the co-ordinator of the **ME COMPUTER department**, and respected **Asst. Professor Sarad Chandra Joshi** for his invaluable guidance and support during the preparation of my thesis proposal. His insights and suggestions have helped me shape my research ideas and direction.

I also appreciate the efforts of **the library staff and technical support team at Everest Engineering College** for providing me with access to the necessary resources and infrastructure to carry out my research.

Finally, I would like to express my gratitude to **my family and friends** for their constant encouragement and motivation. Their unwavering support has been instrumental in my academic and personal growth.

Thank you.

Your Sincerely
Rohit Sharma

Abstract

This case study evaluates DNA storage, holographic storage, and quantum storage using working principle, basic architecture, density and performance, practical limitations, and commercial feasibility over the next twenty years. The study distinguishes classical mass storage from quantum memory, because quantum storage primarily preserves quantum states rather than ordinary files. DNA storage is the strongest candidate for early specialized commercial deployment in archival and cold-data markets, holographic storage remains technically promising but economically uncertain, and quantum storage is most likely to mature as infrastructure for quantum computing and networking.

Table of Contents

Acknowledgement i
Abstract ii
List of Figures iv
1. Introduction 1
2. Backtracking 1
2.1 Core idea 1
2.2 Worked example: N-Queens 1
2.3 Where backtracking fits 3
3. Branch and Bound 3
3.1 Core idea 3
3.2 Worked example: 0/1 Knapsack 4
3.3 Where branch and bound fits 6
4. Randomized Algorithms 6
4.1 Core idea 6
4.2 Worked example: Randomized QuickSort 7
4.3 Where randomized algorithms fit 9
5. Comparison 9
6. Conclusion 9
7. References 10

List of Figures

Figure 1: N-Queen Problem..... 2
Figure 2: Branch and Bound algorithm 4
Figure 3: Randomized Algorithms..... 7

1. Introduction

Many optimization and search problems cannot be solved efficiently by straightforward exhaustive search, because the number of possible solutions grows exponentially with input size. Three algorithmic strategies address this in different ways:

Backtracking builds a solution incrementally and abandons a partial solution as soon as it can't possibly lead to a valid one.

Branch and Bound also explores solutions incrementally, but uses a bound (a best-case or worst-case estimate) to decide which branches are worth exploring at all — useful for optimization problems, not just decision problems.

Randomized algorithms use random choices during execution to achieve good expected performance, avoid worst-case inputs, or simplify an otherwise complex deterministic procedure.

This case study looks at each technique individually, works through a representative problem, and then compares them directly.

2. Backtracking

2.1 Core idea

Backtracking is a refinement of brute-force search. Instead of generating every candidate solution and checking it at the end, it builds a solution one piece at a time. As soon as a partial solution violates a constraint, the algorithm discards it and goes back to try a different choice — hence “backtracking.”

This is usually described as a depth-first search over a state-space tree, with pruning: whole subtrees are skipped once they're known to fail.

2.2 Worked example: N-Queens

Problem: Place N queens on an N×N chessboard so that no two queens attack each other (no shared row, column, or diagonal).

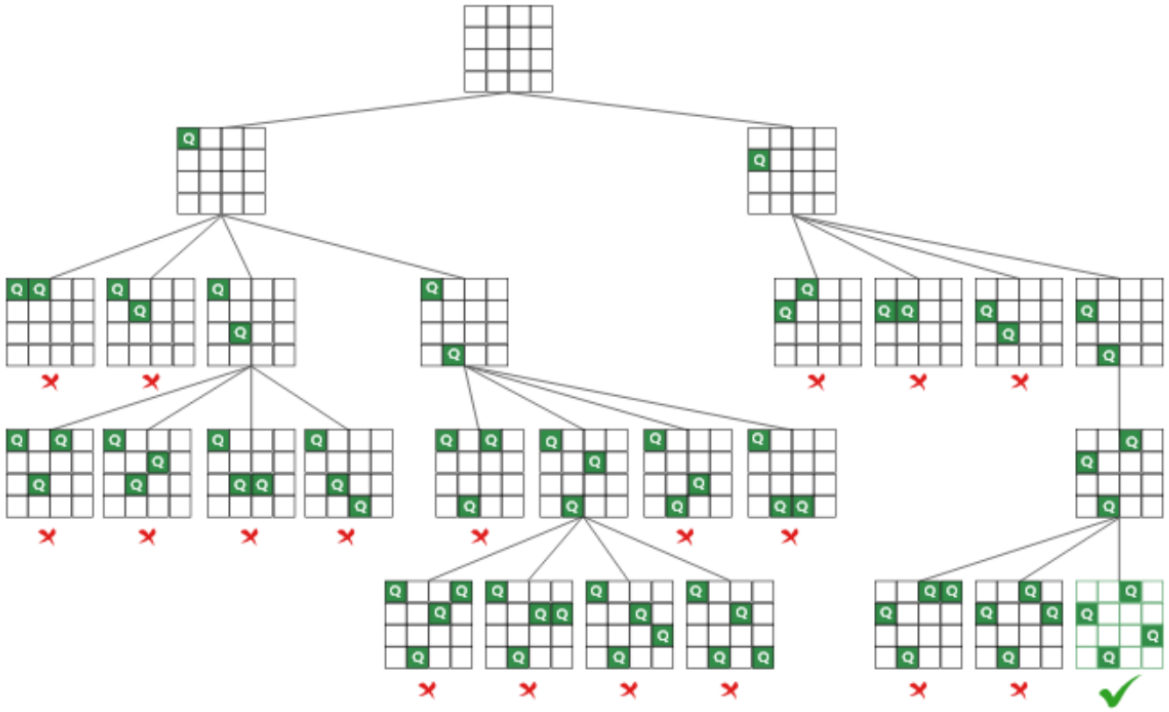


Figure 1: N-Queen Problem

Approach: Place queens column by column. At each column, try each row; if the placement is safe relative to all queens placed so far, recurse into the next column. If no row works, backtrack to the previous column and try its next option.

```
import java.util.ArrayList;
import java.util.List;

public class NQueens {

    private final int n;
    private final int[] columns; // columns[row] = queen's column
    private final List<int[]> solutions = new ArrayList<>();

    public NQueens(int n) {
        this.n = n;
        this.columns = new int[n];
    }

    public List<int[]> solve() {
        place(0);
        return solutions;
    }

    private void place(int row) {
        if (row == n) {
```

```
            solutions.add(columns.clone());
            return;
        }
        for (int col = 0; col < n; col++) {
            if (isSafe(row, col)) {
                columns[row] = col;
                place(row + 1);
                columns[row] = -1; // backtrack
            }
        }
    }

    private boolean isSafe(int row, int col) {
        for (int r = 0; r < row; r++) {
            int c = columns[r];
            if (c == col || Math.abs(c - col) == Math.abs(r - row)) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        NQueens solver = new NQueens(8);
        List<int[]> solutions = solver.solve();
        System.out.println("Solutions found: " + solutions.size());
    }
}
```

Complexity: Worst case is O(N!), since without pruning there are N! ways to place non-attacking-by-row queens. In practice, pruning cuts this down substantially — for N = 8, only 2,057 partial placements are examined out of 8! = 40,320 possible row permutations, and the search finds all 92 solutions.

2.3 Where backtracking fits

Backtracking works well for constraint satisfaction problems — Sudoku, graph coloring, subset-sum, permutation generation — where a solution either exists or doesn't, and partial solutions can be checked for validity cheaply. It doesn't rank solutions by quality; it just finds ones that satisfy the constraints.

3. Branch and Bound

3.1 Core idea

Branch and Bound also searches a state-space tree, but it's built for optimization problems (minimize cost, maximize value) rather than pure constraint satisfaction. At each node, it computes a bound — an estimate of the best possible outcome reachable from that node. If the

bound is worse than the best solution found so far, the entire subtree is pruned, because exploring it further cannot improve the answer.

The three components are:

- Branching: dividing the problem into smaller subproblems.
- Bounding: computing an upper or lower bound for each subproblem.
- Pruning: discarding subproblems whose bound cannot beat the current best solution.

Traversal order matters more here than in plain backtracking. A common choice is best-first search, using a priority queue ordered by bound, so the most promising nodes are expanded first.

3.2 Worked example: 0/1 Knapsack

Problem: Given items with weights and values, and a knapsack of capacity W, choose a subset of items (each item taken 0 or 1 times) to maximize total value without exceeding W.

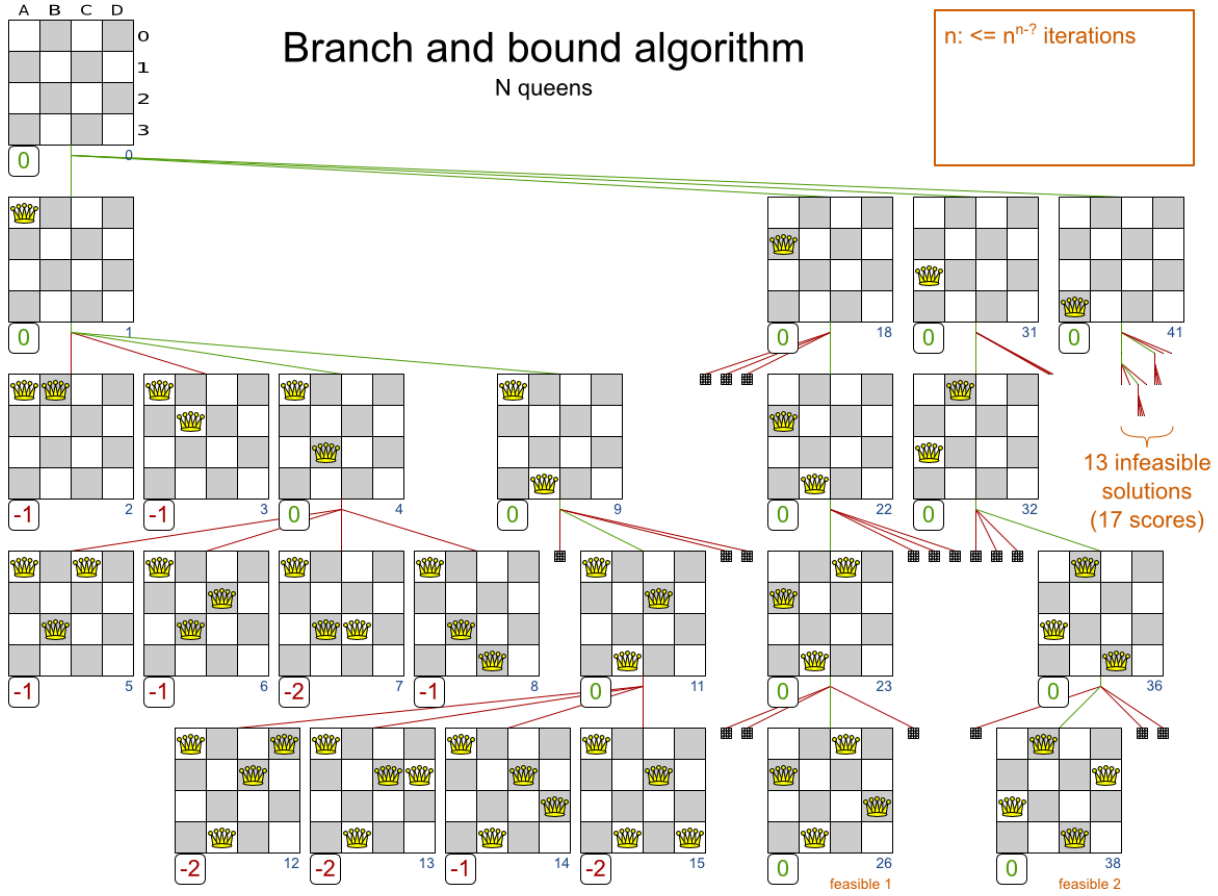


Figure 2: Branch and Bound algorithm

Bounding function: At any node (a partial decision about the first k items), compute an upper bound on achievable value by allowing the remaining items to be taken fractionally, sorted by value-to-weight ratio. This is the same bound used in the fractional knapsack problem and is quick to compute.

```
import java.util.Arrays;
import java.util.Comparator;

public class KnapsackBranchAndBound {

    static class Item {
        final double value;
        final double weight;

        Item(double value, double weight) {
            this.value = value;
            this.weight = weight;
        }
    }

    private final Item[] items;
    private final int n;
    private final double capacity;
    private double bestValue = 0;

    public KnapsackBranchAndBound(Item[] items, double capacity) {
        Item[] sorted = items.clone();
        Comparator<Item> byRatio = Comparator.comparingDouble(i -> i.value / i.weight);
        Arrays.sort(sorted, byRatio.reversed());
        this.items = sorted;
        this.n = sorted.length;
        this.capacity = capacity;
    }

    public double solve() {
        explore(0, 0.0, 0.0);
        return bestValue;
    }

    private double bound(int index, double currentWeight, double currentValue) {
        if (currentWeight >= capacity) {
            return 0;
        }
        double result = currentValue;
        double totalWeight = currentWeight;
        int i = index;
        while (i < n && totalWeight + items[i].weight <= capacity) {
            totalWeight += items[i].weight;
            result += items[i].value;
            i++;
        }
```

```
        if (i < n) {
            result += (capacity - totalWeight) * (items[i].value / items[i].weight);
        }
        return result;
    }

    private void explore(int index, double currentWeight, double currentValue) {
        if (currentWeight <= capacity) {
            bestValue = Math.max(bestValue, currentValue);
        }
        if (index == n || currentWeight >= capacity) {
            return;
        }
        if (bound(index, currentWeight, currentValue) <= bestValue) {
            return; // prune: this branch can't beat what we already have
        }
        Item item = items[index];
        // take item, then skip item
        explore(index + 1, currentWeight + item.weight, currentValue + item.value);
        explore(index + 1, currentWeight, currentValue);
    }

    public static void main(String[] args) {
        Item[] items = {
                new Item(60, 10),
                new Item(100, 20),
                new Item(120, 30)
        };
        KnapsackBranchAndBound solver = new KnapsackBranchAndBound(items, 50);
        System.out.println("Best value: " + solver.solve());
    }
}
```

Complexity: Worst case is still O(2^N), same as brute force, because bounding gives no guarantee in the worst case. In practice, on typical instances, the bound eliminates large parts of the tree early, and running time is far below the worst case.

3.3 Where branch and bound fits

This is the standard technique for combinatorial optimization: the traveling salesman problem, integer linear programming, job scheduling, and the knapsack problem above. It's the right tool whenever there's a numeric objective to optimize and a way to compute a cheap bound on it.

4. Randomized Algorithms

4.1 Core idea

A randomized algorithm makes some of its decisions using random numbers rather than a fixed rule. There are two standard categories:

- Las Vegas algorithms always produce a correct result, but their running time is a random variable. Randomized QuickSort is the standard example.
- Monte Carlo algorithms run in fixed (often guaranteed) time, but have some probability of producing an incorrect result. The Miller-Rabin primality test is a standard example.

The motivation is usually one of: avoiding worst-case inputs that a deterministic algorithm is vulnerable to, simplifying an algorithm that would otherwise need complicated deterministic logic, or getting a fast approximate answer where an exact one is too costly.

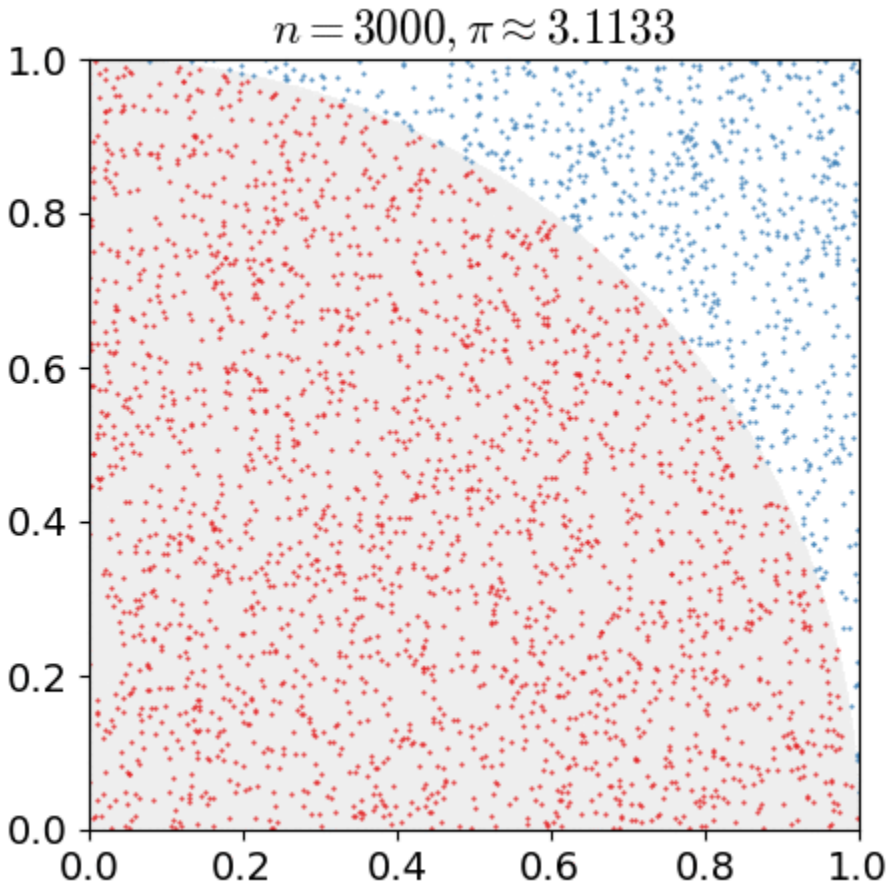


Figure 3: Randomized Algorithms

4.2 Worked example: Randomized QuickSort

Ordinary QuickSort picks a pivot by a fixed rule (e.g., always the first or last element), which means an adversary — or simply an already-sorted input — can force $O(N^2)$ behavior. Randomized QuickSort picks the pivot uniformly at random from the subarray at each step.

```
import java.util.Random;

public class RandomizedQuickSort {

    private static final Random RANDOM = new Random();

    public static void sort(int[] arr, int low, int high) {
        if (low < high) {
            int pivotIndex = low + RANDOM.nextInt(high - low + 1);
            swap(arr, pivotIndex, high);
            int p = partition(arr, low, high);
            sort(arr, low, p - 1);
            sort(arr, p + 1, high);
        }
    }

    private static int partition(int[] arr, int low, int high) {
        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (arr[j] <= pivot) {
                i++;
                swap(arr, i, j);
            }
        }
        swap(arr, i + 1, high);
        return i + 1;
    }

    private static void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }

    public static void main(String[] args) {
        int[] data = {9, 3, 7, 1, 8, 2, 5, 4, 6};
        sort(data, 0, data.length - 1);
        System.out.println(java.util.Arrays.toString(data));
    }
}
```

Complexity: Worst case is still $O(N^2)$, because an unlucky sequence of random choices is always possible. But the expected running time, over the randomness, is O(N log N) for any input — no input can reliably force bad behavior, since the pivot choice doesn't depend on the input's arrangement.

4.3 Where randomized algorithms fit

They're useful when worst-case adversarial inputs are a real concern (hash table design, quicksort, skip lists), when a problem has no known fast deterministic algorithm but a fast randomized one exists (some primality testing, some approximate counting problems), or when simplicity matters and a small failure probability is acceptable.

5. Comparison

Aspect	Backtracking	Branch and Bound	Randomized Algorithms
Problem type	Constraint satisfaction (feasibility)	Optimization	General — feasibility or optimization
Search strategy	Depth-first, prune on constraint violation	Depth-first or best-first, prune on bound	Not tree-search based; relies on random choice
Output	Any/all feasible solutions	Best (optimal) solution	Correct result (Las Vegas) or probably-correct result (Monte Carlo)
Worst-case time	Exponential	Exponential	Depends on algorithm; expected time often polynomial
Determinism	Deterministic	Deterministic	Non-deterministic — output or runtime varies by run
Example problem	N-Queens, Sudoku	0/1 Knapsack, TSP	Randomized QuickSort, primality testing

The key distinction between backtracking and branch and bound is what they're pruning on: backtracking prunes branches that are infeasible, while branch and bound prunes branches that are feasible but provably not better than a known solution. Randomized algorithms are a different category altogether — they don't rely on exhaustive tree search, so their guarantees are stated in terms of probability or expectation rather than a fixed worst case.

6. Conclusion

Backtracking, branch and bound, and randomized algorithms all exist because exhaustive search is too slow for most real problem sizes. Backtracking cuts the search space by rejecting invalid partial solutions early. Branch and bound extends that idea to optimization problems using numeric bounds. Randomized algorithms take a different approach entirely, trading a deterministic guarantee for good expected performance or a controllable error probability. Which

one applies depends on the problem: whether it's a search for any feasible answer, a search for the best answer, or a case where average-case speed and simplicity outweigh the need for a guaranteed worst case.

7. References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] E. Horowitz, S. Sahni, and S. Rajasekaran, *Computer Algorithms/C++*. Summit, NJ, USA: Silicon Press, 1998.

[3] J. Kleinberg and É. Tardos, *Algorithm Design*. Boston, MA, USA: Addison-Wesley, 2006.

[4] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge, U.K.: Cambridge Univ. Press, 1995.

[5] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2011.